Building Map Data for ESP32 Displays

A practical guide to roads, railways, waterways and forest shading using OpenStreetMap, ESA WorldCover and custom binary files

Example platform: LILYGO T-RGB 2.1-inch, ESP32-S3, 480 × 480 display

Adaptable to other ESP32 map projects. The VEC1 and FRST formats described here are specific to the example firmware, not universal map standards.

From geographic data to a fast, offline map

Data sources • extraction • simplification • binary conversion • FFat • debugging • attribution

1. What this guide covers

An embedded map usually does not need full GIS datasets at runtime. Prepare a limited geographic area on a computer, simplify it for the display, convert it to compact binary files, and store them in flash. The ESP32 loads those files into memory and draws the geometry visible in the current viewport.

The example firmware uses chernik.vec for road-like line features and forests.vec for forest polygons. Your own firmware may use different filenames, scales, coordinate systems, memory layouts or rendering methods.

Typical pipeline

```
Choose bounding box
      ↓
Download source data (OSM + optional land cover)
      ↓
Filter features and clip to bounds
      ↓
Simplify geometry and quantize coordinates
      ↓
Encode VEC1 road data and FRST forest data
      ↓
Validate headers, counts, bounds and file sizes
      ↓
Copy files to FFat and inspect Serial output
```

2. Choose a geographic area

Start with a bounding box around the area the device must display. Larger areas create more geometry, larger files, more memory use and longer rendering times.

Example bounds

```
min latitude:   43.10
min longitude:  26.70
max latitude:   44.20
max longitude:  28.00
```

These are example bounds from one project, not universal defaults. Use decimal degrees and confirm the order expected by each tool. Latitude increases northward; longitude increases eastward.

- Begin with a small test area and verify the map appears in the correct place.
- Add a margin around the intended driving area.
- Keep bounds and quantization scales consistent across files that overlay one another.
- Make sure the data covers the viewport around the moving GPS position.

3. Obtain road and line data from OpenStreetMap

Open Overpass Turbo. Zoom to the target area or use an explicit bounding box. This example requests highways, railways and waterways:

```
[out:json][timeout:60];
(
  way["highway"](43.10,26.70,44.20,28.00);
  way["railway"](43.10,26.70,44.20,28.00);
  way["waterway"](43.10,26.70,44.20,28.00);
);
out geom;
```

The Overpass bounding-box order is south, west, north, east. Large queries may time out; reduce the area or request fewer feature classes. The output is OSM JSON, not VEC1, so a separate conversion step is needed.

Feature selection

- Roads: select highway classes useful at the intended zoom. Dense residential and service roads can overwhelm a small display.
- Railways: include only railway types supported by your renderer.
- Waterways: include rivers or streams that your firmware can draw.
- Discard tags and geometry types your converter and firmware do not understand.

4. Add forest / tree-cover data

OSM may contain forest and wood polygons, but coverage varies. ESA WorldCover land-cover data can supplement those areas. The example workflow combined OSM forest polygons with a WorldCover tree-cover supplement.

Start at the official ESA WorldCover portal. Check the exact product year, download access, class definitions and licence for the version you use. WorldCover is raster data; it must be polygonized or otherwise transformed before use by a polygon renderer.

Practical raster-to-polygon workflow

- Clip the raster to the target bounds before polygonizing.
- Select the tree-cover class using the documentation for that exact product version; do not assume class codes are unchanged.
- Dissolve adjacent selected pixels, remove tiny components and simplify boundaries. Polygonizing every pixel independently can create an enormous number of polygons.
- Check how holes are represented. A simple one-ring polygon format may fill holes incorrectly.
- Preview the output on a desktop map before converting it to embedded data.

5. Clip, simplify and quantize geometry

Desktop GIS geometry is often too detailed for a microcontroller. Clip and simplify before converting it to a compact format.

- Reproject to a suitable coordinate system if your simplification tolerance or area filters are in metres.
- Clip features to the chosen bounds; remove empty, invalid or duplicate geometries.
- Simplify lines and forest boundaries for the actual display scale.
- Merge adjacent forest areas where useful; discard polygons too small to be visible.
- Convert coordinates to the exact integer coordinate system expected by the firmware.

Quantization concept

```
qlat = round((latitude  - minLatitude)  * latScale)
qlon = round((longitude - minLongitude) * lonScale)
```

This is conceptual, not a complete converter. Confirm the exact rounding, clipping, coordinate order and scale definitions in your implementation. Quantized values are signed 16-bit integers in the example format, so reject values outside -32768 to 32767 rather than allowing overflow.

6. Example binary formats: VEC1 and FRST

These are custom formats used by the example firmware, not GIS standards. The firmware reads little-endian integer fields, so the converter must write fields in the exact order and byte order shown.

VEC1 road / line file — 32-byte header

```
uint32 magic = 0x56454331  // VEC1 on little-endian systems
int32  minLat_e7
int32  minLon_e7
int32  maxLat_e7
int32  maxLon_e7
uint32 latScale
uint32 lonScale
uint32 segmentCount
```

Each segment follows the header:

```
uint8  tier
uint16 pointCount
repeat pointCount times:
    int16 qlat
    int16 qlon
```

Tier meanings and colours are firmware-specific. If you change them, update the renderer to match.

FRST forest polygon file — 32-byte header

```
uint32 magic = 0x54535246  // FRST on little-endian systems
int32  minLat_e7
int32  minLon_e7
int32  maxLat_e7
int32  maxLon_e7
uint32 latScale
uint32 lonScale
uint32 polygonCount
```

Each polygon follows the header:

```
uint16 pointCount
repeat pointCount times:
    int16 qlat
    int16 qlon
```

The simple FRST layout stores one ring per polygon; it does not explicitly encode holes, IDs or per-polygon metadata. Extending that requires coordinated changes to both converter and firmware.

Required checks

- Magic numbers must match exactly.
- Bounds and latScale/lonScale must match between road and forest files.
- Declared point counts must fit within the remaining file bytes.
- Check signed 16-bit coordinate limits and 16-bit point-count limits.
- PSRAM must be sufficient for the loaded blobs, record tables, framebuffer and forest scratch space.
- The file should end exactly after the final record; investigate trailing or missing bytes.

7. Validate the files on your computer

Do not rely only on whether the device boots. A validation script should read each file from start to finish and report:

- Magic, bounds, scales, record counts and file size.
- Total vertices and maximum vertices in any polygon.
- Any truncated record, overflow, invalid geometry or rejected feature.
- Whether the final parsed offset equals the file size.

Render a preview or inspect the converted geometry in a desktop viewer to catch shifted, mirrored, clipped or incorrectly ordered coordinates. Keep the report alongside the binary files and record the converter version and settings.

8. Copy files to FFat and test

The example firmware expects the following paths at the FFat root:

```
/chernik.vec
/forests.vec
```

Use the upload method appropriate to your board and ESP32 toolchain. Confirm that the selected partition scheme includes FFat and that your uploader writes to the same filesystem mounted by the sketch. Keep a copy of the exact binaries uploaded.

The example firmware treats the road file as mandatory and forest data as optional. Serial output should report successful road loading and, if accepted, the forest file size, polygon count and maximum points per polygon. If forest parsing fails, this sketch continues with roads only.

```
Loading road map...
Road map loaded successfully.
Loading forest map...
Forest file size: ...
Forest polygons in header: ...
FOREST OK: ... polygons, maximum ... points per polygon
Map startup complete.
```

9. Troubleshooting

Road file will not load

Check FFat mount, exact /chernik.vec path, file size, filesystem partition and upload method.

Road file fails to parse

Check VEC1 magic, 32-byte header, record counts, point counts and truncation.

Forest header mismatch

Regenerate FRST using the exact same bounds and quantization scales as VEC1.

Forest shapes are shifted

Check coordinate order, sign, min-bound offsets, quantization and projection.

Huge or malformed forest fills

Inspect invalid polygons, holes, ring structure, clipping and overflow in the conversion pipeline.

Forest parser runs out of memory

Reduce polygon count and maximum vertices per polygon; dissolve, simplify and remove tiny components.

Map draws slowly

Reduce area/detail and profile rendering before changing firmware logic.

Map is blank or features are off-screen

Check bounds, zoom, coordinate order and whether the GPS location is inside the data coverage.

Map jumps while stationary

GPS position jitter can shift a GPS-centred map. Check fix quality and reception before changing the renderer.

10. Attribution and licensing

OpenStreetMap data is made available under the Open Database License (ODbL). Follow current attribution requirements and review share-alike obligations that may apply to a derived database. Appropriate attribution depends on how the map is used and distributed.

OSM copyright and attribution: openstreetmap.org/copyright.

ESA WorldCover has its own product terms and citation/attribution requirements. Check the licence for the exact product and release downloaded; do not assume public access means unrestricted redistribution.

ESA WorldCover portal: worldcover2021.esa.int.

Keep source URLs, download dates, product versions, processing scripts, simplification settings and licence notes with the project.

11. Repeatable checklist

- ☐ Define and record the bounding box.
- ☐ Download only the required OSM feature classes.
- ☐ Confirm land-cover product version, class definitions and licence.
- ☐ Clip, clean, merge and simplify geometry.
- ☐ Quantize coordinates and reject overflow.
- ☐ Write VEC1 and/or FRST in the exact firmware-specific layout.
- ☐ Validate magic, headers, scales, counts, record lengths and end-of-file.
- ☐ Generate a preview and compare with source data.
- ☐ Copy to FFat and archive the exact uploaded files.

- ☐ Inspect Serial output and test at the intended zoom and location.
- ☐ Record converter version and settings so files can be regenerated.

12. Tools and further reading

Overpass Turbo — https://overpass-turbo.eu/

OpenStreetMap copyright and attribution — https://www.openstreetmap.org/copyright

ESA WorldCover — https://worldcover2021.esa.int/

QGIS desktop GIS — https://qgis.org/

GDAL documentation — https://gdal.org/

This guide describes a repeatable workflow, not a turnkey converter. Exact VEC1/FRST output must match the firmware that reads it. Test new files on the bench before relying on them in a vehicle or field installation.